

# MODULE 4

---

## A Perspective on Testing: Basic Definitions

- **Testing**
  - Process of executing a program to find defects and verify that it works correctly.
  - Checks whether the software behaves as expected.
  - Uses a set of inputs and compares the actual output with the expected output.
- **Test**
  - A test is associated with a specific program behavior.
  - Contains a set of inputs and the expected outputs.
  - Used to verify whether the software produces the correct result.
- **Test Case**
  - A uniquely identified test used to verify a specific behavior of the program.
  - Includes:
    - Test Case ID
    - Purpose
    - Preconditions (requirements before execution)
    - Inputs
    - Expected Outputs
    - Postconditions (state after execution)
    - Execution history (optional)
- **Precondition**
  - Conditions that must be satisfied before running a test.
  - Example: User must already be registered before testing login.
- **Input**

- Data provided to the program during testing.
- Example: Username, password, numbers, or commands.
- **Expected Output**
  - The correct result the program should produce for the given inputs.
  - Used to determine whether the test passes or fails.
- **Postcondition**
  - State of the system after the test is completed.
  - Example: User dashboard opens after successful login.
- **Program Behavior**
  - Describes what the software does for a given input.
  - Compared against the specified (expected) behavior during testing.
- **Specified Behavior**
  - The behavior described in the software requirements or specification.
  - Represents what the system is expected to do.
- **Programmed (Observed) Behavior**
  - The behavior actually implemented in the software.
  - May or may not match the specification.
- **Fault of Omission**
  - A required feature or behavior is missing from the program.
  - Example: Forgot to implement password validation.
- **Fault of Commission**
  - An incorrect or unintended feature or behavior is included in the program.
  - Example: Program accepts invalid inputs due to wrong logic.
- **Functional (Black Box) Testing**
  - Tests software based only on inputs and outputs.
  - Does not require knowledge of internal code.
  - Based on software specifications.
  - Advantages:

- Independent of implementation.
    - Test cases can be prepared before coding is complete.
  - Disadvantages:
    - May contain redundant tests.
    - Some parts of the software may remain untested.
  - **Structural (White Box) Testing**
    - Tests software using knowledge of internal code and implementation.
    - Focuses on program structure and logic.
    - Advantages:
      - Finds hidden code errors.
      - Detects dead or unused code.
      - Provides measurable code coverage.
    - Disadvantages:
      - Time-consuming and costly.
      - Requires programming knowledge.
      - Some code paths may still be missed.
  - **Functional vs Structural Testing**
    - Functional testing checks "**What the software does.**"
    - Structural testing checks "**How the software works internally.**"
    - Both approaches are important and complement each other.
    - Combining both provides better test coverage and confidence in software quality.
- 

## Test Cases

- A **test case** is a set of conditions, inputs, and expected outputs used to verify whether a software feature works correctly.
- Each test case has a **unique identity** and is associated with a specific program behavior.

- It helps determine whether the software produces the expected result.
- **Main components of a test case:**
  - **Test Case ID** – Unique identifier for the test case.
  - **Purpose** – Describes what is being tested.
  - **Preconditions** – Conditions that must be true before executing the test.
  - **Inputs** – Data or values given to the program.
  - **Expected Output** – Correct result expected after execution.
  - **Postconditions** – State of the system after the test is completed.
  - **Execution History** – Record of previous executions and results (if maintained).
- **Example Test Case:**
  - Test Case ID: **TC01**
  - Purpose: Verify login functionality.
  - Preconditions: User is already registered.
  - Inputs: Valid username and password.
  - Expected Output: Login successful.
  - Postcondition: User dashboard is displayed.
- **Good test cases should:**
  - Clearly define inputs and expected outputs.
  - Cover both valid and invalid scenarios.
  - Be easy to execute and repeat.
  - Help identify defects in the software.
  - Be independent so they can be run in any order.
- **Importance of test cases:**
  - Ensure software behaves as expected.
  - Improve test coverage.
  - Make testing systematic and repeatable.
  - Help detect bugs before software release.

- Support regression testing after code changes.
- 

## Insights from a Venn Diagram

- Testing is mainly concerned with two views:
  - **Behavioral view** – focuses on **what the program does**.
  - **Structural view** – focuses on **how the program is built**.
- The Venn diagram uses three sets:
  - **S (Specified Behavior)** – Expected behaviors defined in the requirements.
  - **P (Programmed Behavior)** – Behaviors actually implemented in the software.
  - **T (Test Cases)** – Behaviors covered by the test cases.
- The **intersection of S and P** represents the **correct behavior**:
  - These behaviors are both specified and correctly implemented.
- **Fault of Omission**
  - Occurs when a required behavior is in **S** but not in **P**.
  - The software is missing a specified feature or function.
- **Fault of Commission**
  - Occurs when a behavior is in **P** but not in **S**.
  - The software performs an unintended or unspecified action.
- **Insights from the relationship of S, P, and T:**
  - If specified behaviors have **no test cases**, testing is **incomplete**.
  - If test cases cover **unspecified behaviors**, either:
    - The test cases are unnecessary, or
    - The specification is incomplete or incorrect.
  - If programmed behaviors are **not tested**, defects may remain undetected.
- **Goal of testing**
  - Make the common intersection of **Specified Behavior (S)**, **Programmed Behavior (P)**, and **Test Cases (T)** as large as possible.

- Ensure required features are implemented and thoroughly tested.
  - **Test cases can be identified using two approaches:**
    - **Functional (Black Box) Testing** – based on software specifications.
    - **Structural (White Box) Testing** – based on program implementation.
- 

## Identifying Test Cases

- **Test cases** are designed to verify whether the software behaves correctly and to find defects.
- There are **two basic approaches** to identify test cases:
  - **Functional (Black Box) Testing**
  - **Structural (White Box) Testing**

### Functional (Black Box) Approach

- Test cases are created using the **software specification or requirements**.
- Focuses on **what the software should do**.
- Does **not** require knowledge of the internal code.
- Used to check whether the software meets user requirements.
- Helps establish confidence that the system works correctly.

### Structural (White Box) Approach

- Test cases are created using the **program's source code and implementation**.
- Focuses on **how the software is built and executed**.
- Requires knowledge of the internal logic and code.
- Helps find hidden defects and measure code coverage.
- Mainly used to discover faults in the implementation.

## Comparison

- **Functional Testing**
  - Based on specifications.
  - Tests external behavior.

- Ignores internal code.
  - Verifies expected functionality.
  - **Structural Testing**
    - Based on source code.
    - Tests internal logic and execution paths.
    - Requires programming knowledge.
    - Detects implementation errors and uncovered code.
- 

## Levels of Testing

- Testing is performed at different levels to detect defects early and ensure the software works correctly.

### Unit Testing

- Tests **individual functions, methods, or modules** in isolation.
- Performed by developers.
- Focuses on checking the correctness of a single unit of code.
- Mainly used for **defect testing**.

### Component Testing

- Tests **multiple related units combined into a component**.
- Ensures that units work correctly when integrated.
- Focuses on **interfaces and interactions** between units.

### System Testing

- Tests the **entire integrated system** as a whole.
- Verifies that all components work together correctly.
- Focuses on **component interactions**, data flow, and overall system behavior.
- Checks whether the complete system meets its requirements.

### Release Testing

- Performed on a **complete version** of the software before it is released.

- Usually carried out by a **separate testing team**.
- Ensures the product is ready for delivery to users.
- Confirms that the software satisfies functional and non-functional requirements.

## User Testing

- Performed by **actual users or customers** in their own environment.
- Verifies that the software meets user needs and works in real-world conditions.
- Helps identify usability and practical issues.

## Types of User Testing

- **Alpha Testing**
    - Conducted at the developer's site.
    - Users test the software with support from the development team.
  - **Beta Testing**
    - Conducted by selected users in their own environment.
    - Developers are not present during testing.
    - Feedback is used to fix remaining issues.
  - **Acceptance Testing**
    - Performed by the customer to decide whether to accept the software.
    - Confirms that the system satisfies business and user requirements.
    - Commonly used for custom-built software.
- 

## Generalized Pseudocode

- **Generalized pseudocode** is a **language-independent** way of writing program logic.
- It describes the steps of an algorithm without using the syntax of any specific programming language.
- It is easy to read and understand by programmers, testers, and designers.
- It focuses on **logic** rather than programming details.

- It is commonly used to explain software behavior before actual coding begins.
  - **Purpose of generalized pseudocode:**
    - Represents program logic in a simple form.
    - Helps in designing algorithms before implementation.
    - Makes communication easier among developers and testers.
    - Can be translated into any programming language.
  - **Advantages:**
    - Language neutral.
    - Easy to modify and review.
    - Helps identify logical errors early.
    - Useful for creating test cases and flowcharts.
  - **Common examples discussed in testing:**
    - **Triangle Problem** – Classifies a triangle as Equilateral, Isosceles, Scalene, or Not a Triangle based on three side lengths.
    - **NextDate Function** – Computes the next valid calendar date from a given day, month, and year.
    - **Commission Problem** – Calculates a salesperson's commission based on sales rules and input values.
    - Other example applications include **SATM System**, **Currency Converter**, and **Saturn Windshield Wiper Controller**.
- 

## The Triangle Problem

- The Triangle Problem is a common software testing example used to verify program logic.
- The program accepts **three integers** `a`, `b`, and `c` representing the sides of a triangle.
- Each side must satisfy:
  - `1 ≤ a ≤ 200`
  - `1 ≤ b ≤ 200`

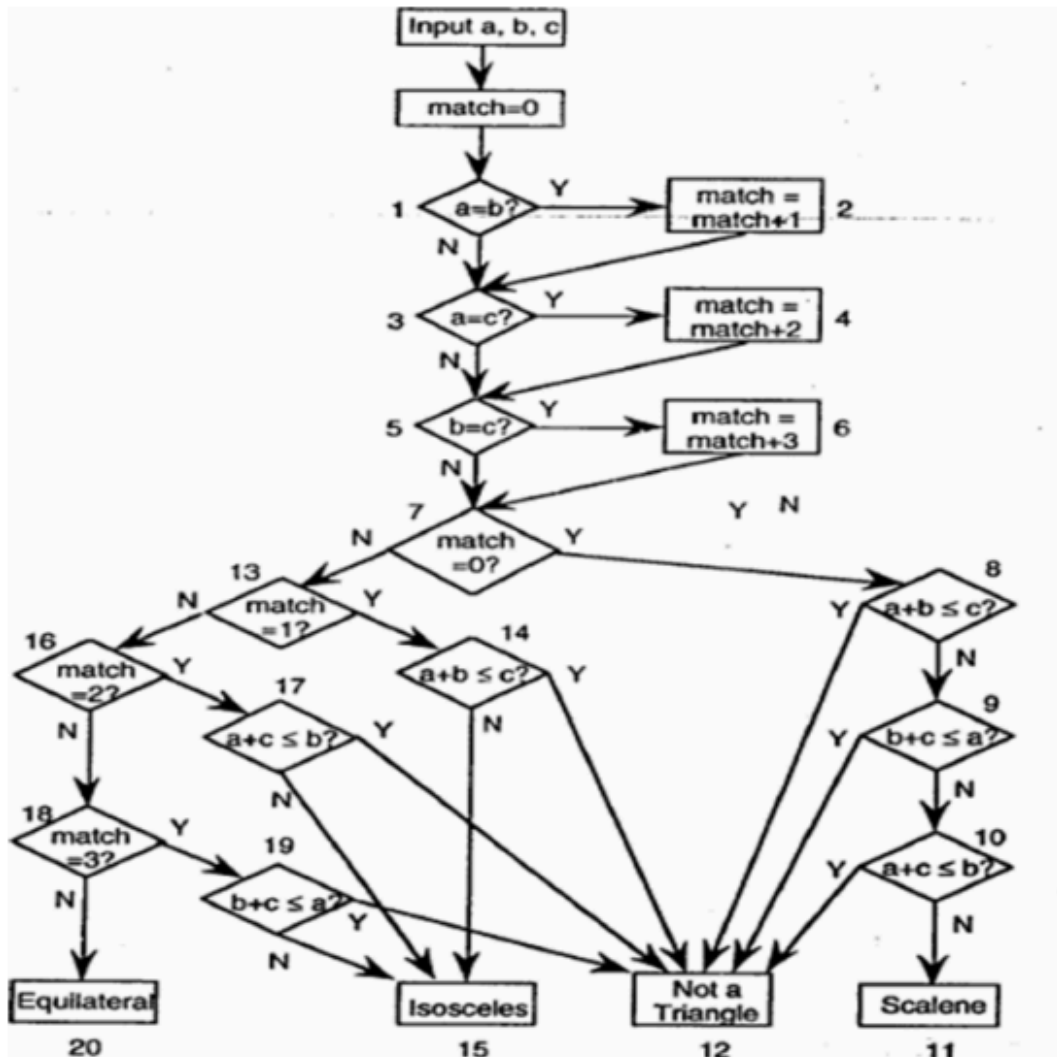
- $1 \leq c \leq 200$
- The program checks the **triangle inequality conditions**:
  - $a < b + c$
  - $b < a + c$
  - $c < a + b$
- If any of these conditions fail, the output is **"Not a Triangle"**.
- Based on the side lengths, the triangle is classified as:
  - **Equilateral** – All three sides are equal ( $a = b = c$ ).
  - **Isosceles** – Exactly two sides are equal.
  - **Scalene** – All three sides are different.
  - **Not a Triangle** – Triangle inequality is not satisfied.
  - **Invalid Input** – One or more side values are outside the allowed range.
- The program first validates the input values.
- It then checks equality between the sides.
- Finally, it applies the triangle inequality rules to determine the correct output.

```

Display message: "Enter 3 integers which are sides of a triangle".
Read inputs a, b, c.
Initialize a variable match = 0.
Check equality of sides:
  If a = b, set match = match + 1.
  If a = c, set match = match + 2.
  If b = c, set match = match + 3.
If match = 0:
  Check triangle inequalities:
    If a + b ≤ c → Output "Not a Triangle".
    Else if b + c ≤ a → Output "Not a Triangle".
    Else if a + c ≤ b → Output "Not a Triangle".
    Else → Output "Scalene".
If match = 1:
  If a + b ≤ c → Output "Not a Triangle".
  Else → Output "Isosceles".
If match = 2:
  If a + c ≤ b → Output "Not a Triangle".
  Else → Output "Isosceles".
If match = 3:
  If b + c ≤ a → Output "Not a Triangle".
  Else → Output "Isosceles".
Otherwise:

```

Output "Equilateral".  
End the program.



| TEST CASE ID | INPUT (A, B, C) | EXPECTED OUTPUT | DESCRIPTION                                      |
|--------------|-----------------|-----------------|--------------------------------------------------|
| 1            | (5, 5, 5)       | Equilateral     | All sides are equal.                             |
| 2            | (5, 5, 8)       | Isosceles       | Exactly two sides are equal.                     |
| 3            | (3, 4, 5)       | Scalene         | No sides are equal.                              |
| 4            | (1, 2, 3)       | Not A Triangle  | Violates triangle inequality ( $a + b \leq c$ ). |
| 5            | (2, 3, 1)       | Not A Triangle  | Violates triangle inequality ( $a + c \leq b$ ). |
| 6            | (3, 1, 2)       | Not A Triangle  | Violates triangle inequality ( $b + c \leq a$ ). |
| 7            | (0, 5, 5)       | Invalid Input   | Side $a$ is out of range ( $a < 1$ ).            |
| 8            | (5, 0, 5)       | Invalid Input   | Side $b$ is out of range ( $b < 1$ ).            |
| 9            | (5, 5, 0)       | Invalid Input   | Side $c$ is out of range ( $c < 1$ ).            |
| 10           | (201, 5, 5)     | Invalid Input   | Side $a$ is out of range ( $a > 200$ ).          |
| 11           | (5, 201, 5)     | Invalid Input   | Side $b$ is out of range ( $b > 200$ ).          |
| 12           | (5, 5, 201)     | Invalid Input   | Side $c$ is out of range ( $c > 200$ ).          |

- The Triangle Problem is widely used for:
    - Testing conditional statements.
    - Designing black-box and white-box test cases.
    - Applying boundary value analysis and equivalence class testing.
    - Verifying that all possible input combinations are handled correctly.
- 

## The Next Date Function

- The **Next Date Function** calculates the calendar date immediately following a given date.
- It takes **month (MM)**, **day (DD)**, and **year (YYYY)** as input.
- It returns the **next valid date** as output.
- It is commonly used in calendar systems, scheduling software, reservation systems, banking applications, and payroll systems.
- The input date must satisfy:
  - $1 \leq \text{month} \leq 12$
  - $1 \leq \text{day} \leq 31$
  - $1812 \leq \text{year} \leq 2012$  (or the specified valid range in the problem).
- Before computing the next date, the function validates the input.
- If any value is outside the valid range, it reports an **invalid input** and requests a valid date.
- The function handles different cases:
  - **31-day months (January, March, May, July, August, October):**
    - If the day is less than 31, increment the day.
    - Otherwise, set day to 1 and move to the next month.
  - **30-day months (April, June, September, November):**
    - If the day is less than 30, increment the day.
    - If the day is 30, set day to 1 and move to the next month.
  - **December:**

- If the day is less than 31, increment the day.
  - If the day is 31, set day to 1, month to January, and increment the year.
- **February:**
  - In a leap year, February has 29 days.
  - In a non-leap year, February has 28 days.
  - The function correctly handles transitions from February to March.
- Typical test cases include:
  - December 31 → January 1 of the next year.
  - February 28 → February 29 in a leap year.
  - February 29 → March 1 in a leap year.
  - February 28 → March 1 in a non-leap year.
  - April 30 → May 1.
  - January 31 → February 1.

Read the input values: month, day, and year.

Validate the input:

```
1 ≤ month ≤ 12
1 ≤ day ≤ 31
1812 ≤ year ≤ 2012
```

If any condition fails, display an error message and request valid input.

Case 1: 31-day months (January, March, May, July, August, October)

```
If day < 31:
    tomorrowDay = day + 1
Else:
    tomorrowDay = 1
    tomorrowMonth = month + 1
```

Case 2: 30-day months (April, June, September, November)

```
If day < 30:
    tomorrowDay = day + 1
Else if day = 30:
    tomorrowDay = 1
    tomorrowMonth = month + 1
Else:
    Output "Invalid Input Date"
```

Case 3: December

```
If day < 31:
    tomorrowDay = day + 1
```

```

Else:
    tomorrowDay = 1
    tomorrowMonth = 1
    tomorrowYear = year + 1

Case 4: February
If day < 28:
    tomorrowDay = day + 1
Else if day = 28:
    If it is a leap year:
        tomorrowDay = 29
Else:
    tomorrowDay = 1
    tomorrowMonth = 3
Else if day = 29:
    If it is a leap year:
        tomorrowDay = 1
        tomorrowMonth = 3
Else:
    Output "Invalid Input Date"

Return the values of tomorrowDay, tomorrowMonth, & tomorrowYear as
next valid date.

```

- Test Cases for NextDate Function

| Test Case ID | Input (Month, Day, Year) | Expected Output | Description              |
|--------------|--------------------------|-----------------|--------------------------|
| TC1          | (12,31,2020)             | (1,1,2021)      | December 31 to January 1 |
| TC2          | (2,28,2020)              | (2,29,2020)     | Leap year transition     |
| TC3          | (2,29,2020)              | (3,1,2020)      | Leap day to March 1      |
| TC4          | (2,28,2019)              | (3,1,2019)      | Non-leap year transition |
| TC5          | (4,30,2021)              | (5,1,2021)      | April 30 to May 1        |
| TC6          | (1,31,2021)              | (2,1,2021)      | January 31 to February 1 |
| TC7          | (6,30,2021)              | (7,1,2021)      | June 30 to July 1        |

## The Commission Problem

- The Commission Problem is a classic software testing example used to test programs that calculate sales commission.
- A salesperson sells **locks, stocks, and barrels** to make rifles.
- Prices of the items are:
  - **Lock = \$45**

- **Stock = \$30**
  - **Barrel = \$25**
  - Sales constraints:
    - At least **one complete rifle** must be sold each month.
    - Maximum sales allowed:
      - **70 locks**
      - **80 stocks**
      - **90 barrels**
    - A salesperson can visit **1 to 10 towns** in a month.
  - Commission is calculated as:
    - **10%** on sales up to **\$1000**
    - **15%** on the next **\$800** (from \$1001 to \$1800)
    - **20%** on sales **above \$1800**
  - The program should:
    - Accept the number of locks, stocks, and barrels sold.
    - Calculate the total sales amount.
    - Compute the commission based on the commission slabs.
    - Reject invalid inputs that exceed the allowed limits.
  - The Commission Problem is commonly used to:
    - Test arithmetic calculations.
    - Verify conditional logic and decision making.
    - Apply **Boundary Value Analysis, Equivalence Class Testing, and Decision Table Testing**.
    - Check correctness at important commission boundaries such as **\$1000** and **\$1800**.
- 

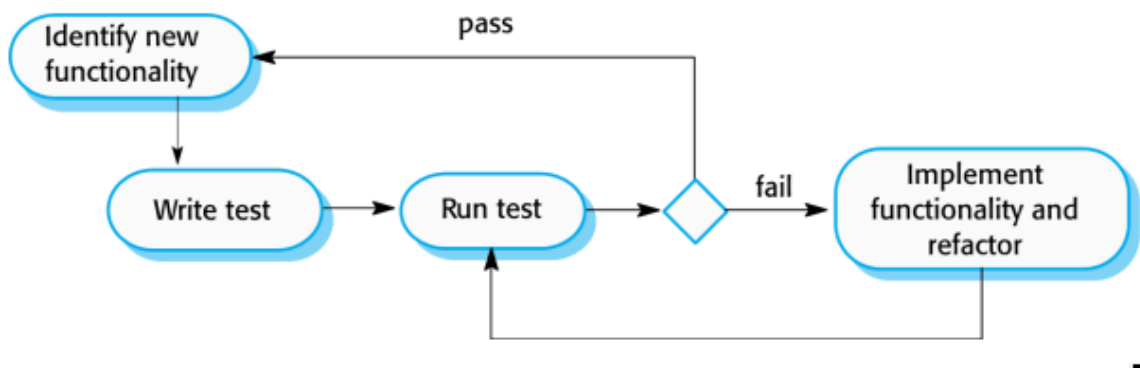
## Development Testing

- Development testing is testing performed by the **team developing the software**.
  - Its main purpose is to **find bugs and defects during development** before the software is released.
  - It helps ensure that individual parts of the software work correctly and interact properly.
  - Development testing consists of three levels:
    - **Unit Testing**
      - Tests individual functions, methods, or classes in isolation.
      - Focuses on the correctness of a single unit.
      - It is mainly a defect testing process.
    - **Component Testing**
      - Tests a group of integrated units or modules.
      - Ensures that components work correctly together.
      - Focuses on interfaces and interactions between units.
    - **System Testing**
      - Tests the complete integrated system.
      - Verifies that all components work together as expected.
      - Focuses on interactions between components and overall system behavior.
  - Development testing is performed continuously throughout software development.
  - It helps detect defects early, reducing the cost and effort required to fix them later.
  - It improves software quality before release to customers.
- 

## Test-Driven Development (TDD)

- Test-Driven Development (TDD) is a software development approach in which **tests are written before the actual code**.
- Testing and coding are performed together in small steps.

- The goal is to write code that passes the predefined tests.
- **TDD process:**
  - Identify a small piece of functionality to implement.
  - Write an automated test for that functionality.
  - Run the test and confirm that it fails (because the code is not yet written).
  - Write the minimum code needed to make the test pass.
  - Run all tests again to verify they pass.
  - Repeat the process for the next functionality.



- **Benefits of TDD:**
  - Every piece of code has at least one associated test.
  - A growing test suite supports **regression testing**.
  - Debugging becomes easier because failures usually occur in newly added code.
  - The tests act as documentation by describing what the code is expected to do.
- **Regression Testing in TDD:**
  - All existing tests are re-run after every code change.
  - Ensures that new changes do not break previously working functionality.
  - Automated testing makes frequent regression testing practical.

## Release Testing

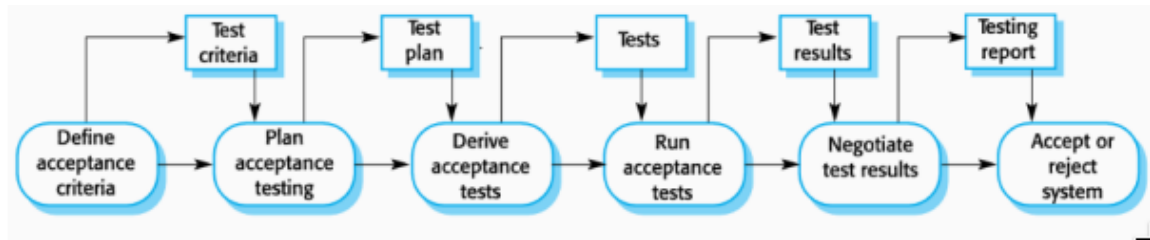
- Release testing is performed on a **complete version of the software** before it is released to users.
  - It is usually carried out by a **separate testing team** that was not involved in development.
  - The main goal is to verify that the software is ready for external use and meets its specified requirements.
  - Release testing is a form of **system testing**.
  - It focuses on **validation testing**, ensuring that the software behaves as expected from the user's point of view.
  - Unlike development testing, which aims to find defects, release testing checks whether the product is suitable for delivery to customers.
  - Common activities in release testing include:
    - Verifying functional requirements.
    - Checking system performance and reliability.
    - Ensuring all major features work correctly.
    - Confirming the software is stable enough for release.
  - Release testing may include:
    - **Requirements-based testing** – creating tests for each software requirement.
    - **Scenario testing** – testing the software using realistic user scenarios.
    - **Performance testing** – checking system behavior under different workloads.
    - **Stress testing** – deliberately overloading the system to observe its behavior under extreme conditions.
  - A successful release test indicates that the software satisfies its requirements and is ready to be deployed to users.
- 

## User Testing

- User testing is a stage of software testing where **users or customers test the system** and provide feedback.

- It is performed after development and release testing.
- The main goal is to ensure that the software works correctly in the **real user environment**.
- It helps evaluate the system's **reliability, performance, usability, and robustness**.
- User testing is important because:
  - Real-world conditions may differ from the testing environment.
  - Users may discover issues that developers and testers missed.
  - It confirms that the software meets user needs and expectations.
- **Types of User Testing:**
  - **Alpha Testing**
    - Performed at the developer's site.
    - Users work with the development team.
    - Developers observe users and collect feedback.
  - **Beta Testing**
    - A release version is given to selected users.
    - Users test the software in their own environment.
    - They report bugs and issues to the developers.
    - Developers are not present during testing.
  - **Acceptance Testing**
    - Performed by customers to decide whether the software is ready for deployment.
    - Mainly used for custom-built systems.
    - Confirms that the software satisfies customer requirements.
- **Stages of Acceptance Testing:**
  - Define acceptance criteria.
  - Plan acceptance testing.
  - Derive acceptance tests.

- Run acceptance tests.
- Negotiate test results.
- Accept or reject the system.



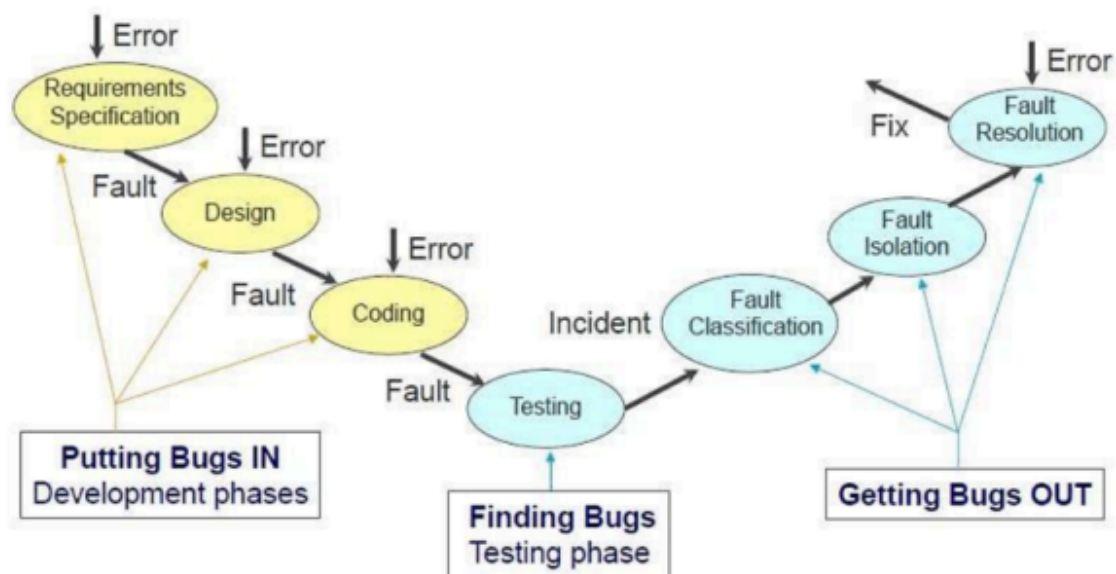
## Error and Fault Taxonomies

- **Taxonomy** means the classification or grouping of errors and faults based on their characteristics.
- It helps developers and testers understand defects and improve software quality.
- **Process**
  - Refers to **how software is developed**.
  - Includes activities like requirements analysis, design, coding, and testing.
- **Product**
  - Refers to the **final software** produced by the development process.
- **Software Quality Assurance (SQA)**
  - Focuses on **improving the development process** to reduce errors.
  - Aims to prevent defects before they occur.
  - Is more concerned with reducing human errors during software development.
- **Testing**
  - Focuses on **finding faults in the software product**.
  - Detects defects that are already present in the program.
- **Faults can be classified into different types:**
  - **Faults by Severity** – Classified based on how seriously they affect the software.

- **Input/Output (I/O) Faults** – Errors related to incorrect input handling or wrong output generation.
- **Logic Faults** – Caused by incorrect program logic or decision-making.
- **Computation Faults** – Errors in calculations or data processing.
- **Interface Faults** – Problems in communication between modules, components, or systems.

## Testing Life Cycle

- The **Testing Life Cycle** explains how bugs are introduced, detected, and removed during software development.



- It consists of three main phases:

### Putting Bugs In (Development Phase)

- Errors may occur during **requirements specification**.
- These errors introduce **faults in the design**.
- Design faults can lead to **coding faults**.
- Human mistakes during development are the main source of software defects.

### Finding Bugs (Testing Phase)

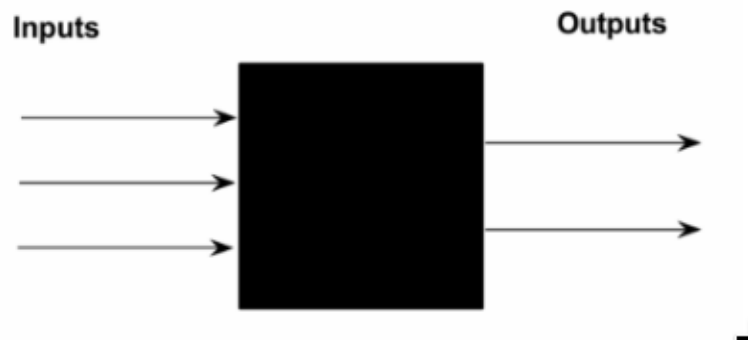
- The software is tested using different test cases.
- Testing helps identify **failures** and **incidents** in the software.
- It reveals defects that cause incorrect behavior or unexpected results.

## Getting Bugs Out (Debugging Phase)

- The detected faults are analyzed and classified.
  - **Fault isolation** is performed to identify the exact cause of the problem.
  - **Fault resolution** is carried out by fixing the defect in the code.
  - The corrected software is tested again to ensure the issue is resolved.
  - The overall flow of defects in the testing life cycle is:
    - **Error** → **Fault** → **Failure** → **Incident**
    - A human **error** introduces a **fault**.
    - When the fault is executed, it causes a **failure**.
    - The failure is observed by the user as an **incident**.
- 

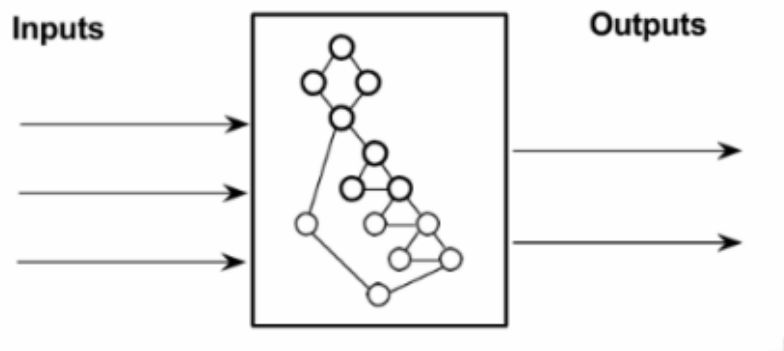
## Black Box Testing

- Black Box Testing is a testing method in which the **tester does not know the internal design or source code** of the software.
- The software is tested by giving **inputs** and checking the **outputs**.
- It focuses on **functionality** rather than implementation.
- It is also called **Functional Testing** or **External Testing**.
- Programming knowledge is **not required**.
- It is mainly performed by **testers**.
- It is commonly used in **System Testing** and **Acceptance Testing**.
- Common techniques include:
  - Equivalence Class Partitioning
  - Boundary Value Analysis
  - Decision Table Testing



## White Box Testing

- White Box Testing is a testing method in which the **tester has complete knowledge of the internal structure, design, and source code** of the software.
- It focuses on **internal logic, code structure, and execution paths**.
- It is also called **Structural Testing** or **Clear Box Testing**.
- Programming knowledge is **required**.
- It is usually performed by **developers**.
- It is commonly used in **Unit Testing** and **Integration Testing**.
- Common techniques include:
  - Statement Coverage
  - Branch Coverage
  - Path Coverage



## Difference Between Black Box & White Box Testing

| Black Box Testing                                             | White Box Testing                                            |
|---------------------------------------------------------------|--------------------------------------------------------------|
| Internal structure is <b>not known</b> to the tester.         | Internal structure is <b>known</b> to the tester.            |
| Focuses on <b>functionality</b> of the software.              | Focuses on <b>internal logic and code</b> .                  |
| Tests inputs and expected outputs.                            | Tests code, control flow, and execution paths.               |
| Programming knowledge is <b>not required</b> .                | Programming knowledge is <b>required</b> .                   |
| Implementation knowledge is <b>not required</b> .             | Implementation knowledge is <b>required</b> .                |
| Mainly performed by <b>testers</b> .                          | Usually performed by <b>developers</b> .                     |
| Used in <b>System Testing</b> and <b>Acceptance Testing</b> . | Used in <b>Unit Testing</b> and <b>Integration Testing</b> . |
| Also called <b>Functional Testing</b> .                       | Also called <b>Structural Testing</b> .                      |

## Equivalence Class Partitioning (ECP)

- Equivalence Class Partitioning is a **black box testing technique** used to reduce the number of test cases.
- It divides the input data into **equivalence classes (partitions)**.
- All values within the same class are expected to be processed in the same way.
- Instead of testing every possible input, **one representative value** is selected from each class.
- There are two types of equivalence classes:
  - **Valid Equivalence Class** – Contains inputs that should be accepted by the program.
  - **Invalid Equivalence Class** – Contains inputs that should be rejected by the program.
- The main idea is:
  - If one value in an equivalence class works correctly, the other values in that class are also expected to work correctly.
  - Similarly, if one invalid value is rejected, other values in that invalid class are expected to be rejected.
- **Advantages:**
  - Reduces the number of test cases.

- Saves testing time and effort.
  - Provides good test coverage with fewer tests.
  - Eliminates redundant testing.
  - **Example: Age field (18–60)**
    - Valid class: **18 to 60**
    - Invalid class: **Less than 18**
    - Invalid class: **Greater than 60**
    - Representative test values: **17, 30, 61**
  - **Example: NextDate Function**
    - Valid month: **1 to 12**
    - Invalid month: **Less than 1**
    - Invalid month: **Greater than 12**
    - Valid day: **1 to 31**
    - Invalid day: **Less than 1**
    - Invalid day: **Greater than 31**
    - Valid year: **1812 to 2012**
    - Invalid year: **Less than 1812**
    - Invalid year: **Greater than 2012.**
- 

## Boundary Value Analysis (BVA)

- Boundary Value Analysis is a **black box testing technique** used to identify test cases at the **boundaries of the input range**.
- It is based on the idea that **errors are more likely to occur at the minimum and maximum values** of an input than in the middle.
- Instead of testing every value, it tests values **at and near the boundaries**.
- For each input variable, the following values are typically selected:
  - Minimum value
  - Just above the minimum

- A nominal (normal) value
  - Just below the maximum
  - Maximum value
  - Boundary Value Analysis is useful for testing:
    - Numeric ranges
    - Input limits
    - Output ranges
    - Loop counters and array indices
  - **Advantages:**
    - Finds defects that occur at boundary conditions.
    - Requires fewer test cases than exhaustive testing.
    - Simple and effective for range-based inputs.
  - **Limitations:**
    - Assumes input variables are independent.
    - May miss errors caused by combinations of inputs.
    - Does not consider the program's internal logic or meaning of the data.
  - **Example: Age field with valid range 18–60**
    - Test values:
      - 17 (just below minimum)
      - 18 (minimum)
      - 19 (just above minimum)
      - 59 (just below maximum)
      - 60 (maximum)
      - 61 (just above maximum)
  - Boundary Value Analysis can also be applied to **output values** and **internal program variables** such as loop counters and pointers.
- 

## Decision Table Testing

- Decision Table Testing is a **black box testing technique** used to test combinations of different input conditions and the corresponding actions.
- It represents complex business logic in the form of a **table**.
- It helps ensure that **all possible combinations of conditions are tested**.
- It reduces the chance of missing important test cases.
- A decision table consists of two parts:
  - **Conditions** – The input conditions or rules.
  - **Actions** – The outputs or operations performed based on those conditions.
- Each column in the table represents a **rule** or **test case**.
- Conditions are usually represented using values such as **True (T)** or **False (F)**.
- The number of possible combinations depends on the number of conditions.
  - Example:
    - 2 conditions with 2 possible values each →  $2 \times 2 = 4$  **combinations**
    - 3 conditions with 3 possible values each →  $3 \times 3 = 9$  **combinations**.
- **Advantages:**
  - Covers all combinations of input conditions.
  - Prevents missing important test cases.
  - Easy to understand and document complex decision logic.
  - Useful for systems with multiple business rules.
- **Applications:**
  - Triangle Problem
  - NextDate Function
  - Commission Problem
  - Business rule validation and decision-making systems.

---

## Statement Coverage

- Statement Coverage is a **white box testing technique**.

- It measures whether **every executable statement in the program has been executed at least once** during testing.
- The goal is to ensure that no statement in the code is left untested.
- It is expressed as:  

$$\text{Statement Coverage} = (\text{Number of executed statements} / \text{Total number of statements}) \times 100\%$$
- **Advantages:**
  - Simple to measure.
  - Identifies statements that were never executed.
  - Improves basic code coverage.
- **Limitation:**
  - Does not guarantee that every decision or branch in the program has been tested.

## Branch Coverage

- Branch Coverage is a **white box testing technique** that checks whether **every branch (True and False outcome of each decision)** has been executed.
- It ensures that all decision points in the program are tested.
- It is expressed as:  

$$\text{Branch Coverage} = (\text{Number of executed branches} / \text{Total number of branches}) \times 100\%$$
- **Advantages:**
  - Provides better coverage than statement coverage.
  - Detects errors in decision-making logic.
  - Ensures both outcomes of conditional statements are tested.
- **Limitation:**
  - Does not guarantee that every possible execution path has been covered.

## Path Coverage

- Path Coverage is a **white box testing technique** that ensures **every possible execution path** through the program is tested at least once.
  - An execution path is the sequence of statements and branches followed from the start to the end of the program.
  - It is the most comprehensive of the three coverage techniques.
  - **Advantages:**
    - Provides the highest level of code coverage.
    - Detects defects caused by different combinations of decisions.
    - Improves confidence in program correctness.
  - **Limitation:**
    - Can require a very large number of test cases for complex programs.
    - May become impractical when there are many loops or conditions.
  - **Comparison:**
    - **Statement Coverage:** Tests every statement.
    - **Branch Coverage:** Tests every decision outcome (True/False).
    - **Path Coverage:** Tests every possible execution path through the program.
-